

CLAUDE

CLAUDE.md

INHERITED FROM constitution/CLAUDE.md

All rules in constitution/CLAUDE.md (and the constitution/Constitution.md it references) apply unconditionally. Project-specific rules below extend them — they do NOT weaken any universal clause. When this file disagrees with the constitution submodule, the constitution wins.

@constitution/CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

Project

Universal multi-format, multi-language ebook translation system written in Go. Module: `digital.vasic.translator` (Go 1.25.2). Supports FB2, EPUB, TXT, HTML, PDF, DOCX input/output with multiple LLM providers (OpenAI, Anthropic, Zhipu, DeepSeek, Qwen, Gemini, Ollama, LlamaCpp), plus an SSH-based distributed worker system and a WebSocket monitoring server.

Current app version: 2.3.0 (see VERSION). Makefile references 3.0.0 — treat VERSION as authoritative unless told otherwise.

There is extensive reference-level documentation in Documentation/ (notably Documentation/AGENTS.md and Documentation/ARCHITECTURE.md). Documentation/CLAUDE.md describes a legacy Python implementation and is **not** authoritative for the current Go codebase — prefer this file and Documentation/AGENTS.md.

Commands

Build, test, lint, run — all via Makefile:

```
make deps          # go mod download + tidy
make build         # builds grpc-server, api-server, unified-
                  translator into ./build/
```

```

make build-all
    # cross-compile Linux/macOS/Windows (amd64 + arm64)
    into ./dist/
make test          # go test -v ./...
make test-coverage # go test -v -cover ./...
make fmt           # go fmt ./...
make vet           # go vet ./...
make quick-test    # fmt + vet + test
make dev           # runs grpc-server (:50051) and api-server
                   (:8080) in debug
make run-system    # builds then runs grpc + api
make docker-build / docker-run

```

Lint (not in Makefile — invoke directly): `golangci-lint` run using `.golangci.yml` (local import prefix: `digital.vasic.translator`; disables-all then enables a curated set including `gosec`, `gocritic`, `gofumpt`, `revive`, `lll@140`).

Single-test patterns:

```

go test -v ./pkg/translator
go test -v -run TestFunctionName ./pkg/distributed

```

TLS certs for HTTPS/HTTP3 live in `certs/` (`server.crt`, `server.key`) and are required by the API server.

Entry points (cmd/)

The codebase has **many** cmd binaries — pick the right one:

- `cmd/unified-translator/` — **primary** CLI with full provider support (API / local llama.cpp / SSH / monitoring). Flags: `-input/-i`, `-output/-o`, `-source-lang`, `-target-lang`, `-script {cyrillic,latin}`, `-provider {openai,anthropic,zhipu,deepseek,qwen,gemini,ollama,llamacpp,ssh}`, `-model`, `-api-key`, `-base-url`, `-temperature`, `-max-tokens`, `-timeout`, `-ssh-host/-ssh-user/-ssh-password/-ssh-port`, `-llama-binary/-llama-model/-context-size`, `-workers`, `-chunk-size`, `-concurrency`, `-verify`, `-monitoring`, `-monitoring-port`.
- `cmd/grpc-server/` — gRPC service on `:50051` (proto: `pkg/grpc/translator.proto`).
- `cmd/api-server/` + `cmd/server/` — REST/WebSocket API on `:8080` (HTTP/3 capable).
- `cmd/monitor-server/` — WebSocket monitoring hub on `:8090`; dashboards: `monitor.html`, `enhanced-monitor.html`, `/monitor` endpoint.
- `cmd/translate-ssh/` — standalone SSH worker; runs on remote hosts and is invoked by the coordinator.
- `cmd/preparation-translator/` — pre-translation analysis pass (characters, terminology, culture) that produces an analysis JSON consumed by the main translator.
- `cmd/markdown-translator/` — EPUB[?] Markdown workflow for easier manual review.
- `cmd/ebook-translator/`, `cmd/cli/`, `cmd/translator/` — older / specialized CLIs.
- `cmd/deployment/` — deployment management tool.

There are also many prebuilt binaries (10–30 MB each) and `demo-*.go` scripts at the repo root from prior sessions. They are not the source of truth — prefer `cmd/` and `pkg/`.

Architecture

Translation pipeline

1. **Format detection** (`pkg/format/`) → 2. **Parsing** (`pkg/ebook/{fb2,epub,docx,html}_parser.go`) → 3. **Translation** (`pkg/translator/`) → 4. **Output** (format-specific writers, e.g. `pkg/ebook/epub_writer.go`).

FB2 handling is split: `pkg/fb2/parser.go` for FB2-specific XML logic, `pkg/ebook/fb2_parser.go` for integration into the universal pipeline. FB2 XML namespaces (<http://www.gribuser.ru/xml/fictionbook/2.0>, `xlink`) must be preserved, and translation must handle element text, children recursively, **and** tail text.

LLM provider layer

All providers in `pkg/translator/llm/` implement a single `LLMClient` interface (`Translate(ctx, text, prompt) (string, error) + GetProviderName()`). Construction goes through a factory (`NewLLMTranslator`) that validates `provider+model`. Per-provider files: `openai.go`, `anthropic.go`, `zhipu.go`, `deepseek.go`, `qwen.go`, `gemini.go`, `ollama.go`, `llamacpp.go`, `llamacpp_provider.go`, plus `mock.go` for tests.

Event-driven core

`pkg/events/events.go` is a central thread-safe pub/sub bus. Event constants: `EventTranslationStarted/Progress/Completed/Error`, `EventConversionStarted/Progress/Completed/Error`. Components emit via helpers like `EmitProgress(bus, sessionID, msg, data)`. `pkg/websocket/hub.go` subscribes to all events and fans them out to connected dashboard clients, filtering by session ID. When adding a new lifecycle stage, emit events on the bus rather than wiring direct callbacks — the dashboard picks them up automatically.

Distributed / SSH workers

`pkg/distributed/` coordinates remote workers over SSH:

- `coordinator.go` — work distribution and aggregation
- `ssh_pool.go` — pooled SSH connections
- `pairing.go` — worker discovery & handshake
- `fallback.go` — graceful degradation when workers fail
- `version_manager.go` — enforces version sync across workers (they must all run the same build; mismatches trigger rollback/update flows)
- `performance.go`, `security.go` — instrumentation and hardening

Worker-side entry point is `cmd/translate-ssh/`. SSH configs live in `internal/working/config.worker*.json` and `internal/working/config.distributed*.json`. Distributed mode is opt-in via the `distributed.enabled` flag in `config.json`.

Storage & caching

`pkg/storage/` abstracts PostgreSQL, Redis, and SQLite (driver deps are in `go.mod`). Translation cache is keyed by `(text, context)`. Reference translations pre-seed high-quality entries.

Other notable packages

- `pkg/preparation/` — content/character/terminology/culture analysis phase (config in `config.json` under `preparation`)
- `pkg/verification/` — multi-pass quality verification and polishing
- `pkg/script/` — Serbian Cyrillic[?]Latin conversion
- `pkg/markdown/` — EPUB→Markdown→EPUB workflow
- `pkg/security/` — JWT auth, rate limiting (CORS is **server-layer**, not here: `cmd/server/main.go corsMiddleware + internal/config/CORSOrigins`)
- `pkg/batch/`, `pkg/coordination/` — batch & multi-LLM coordination
- `pkg/hardware/` — hardware detection for perf tuning
- `pkg/progress/`, `pkg/report/`, `pkg/logger/`, `pkg/version/`

Configuration

Main config: `config.json` (root). Example/provider-specific configs: `internal/working/config*.json`. Sections: `server`, `security` (JWT, rate limit, CORS), `translation` (provider, model, cache, concurrency, per-provider subsection), `preparation`, `distributed` (workers map, SSH timeouts, health checks), `logging`.

API keys come from **environment variables** only — never commit them. Relevant vars: `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `DEEPSEEK_API_KEY`, `ZHIPU_API_KEY`, `QWEN_API_KEY`, `GEMINI_API_KEY`; SSH: `SSH_WORKER_{HOST,USER,PASSWORD,PORT,REMOTE_DIR}`; `server`: `MONITOR_SERVER_PORT`, `LOG_LEVEL`. `.gitignore` already excludes `.env*`, `config_with_keys.json`, `api_keys.json`, `secrets.*`, `**/qwen_credentials.json`, `.qwen/`, `.translator/`.

Testing layout

Tests live in two places — be aware of both:

- **Alongside source** in each `pkg/*` (standard `_test.go` files, the majority of tests).
- `test/` — cross-cutting suites organized as `unit/`, `integration/`, `e2e/`, `performance/`, `stress/`, `security/`, `distributed/`, with shared `mocks/`, `utils/helpers.go`, and `fixtures/`.

- `tests/websocket_monitoring_test.go` — top-level WebSocket integration test (note: `test/` and `tests/` are different directories).

Use build tags for selective execution: `//go:build integration`, `//go:build e2e`. Prefer table-driven tests; naming: `TestFunctionName_Scenario`. Coverage artifacts (`coverage*.out`, `coverage.html`) are committed to the repo — regenerate them, don't hand-edit.

Conventions

- Module-local imports: `digital.vasic.translator/...` (enforced by `goimports` local-prefixes in `.golangci.yml`).
- Error wrapping: `fmt.Errorf("...: %w", err)`.
- Prefer `any` over `interface{}`.
- lll line length cap: 140.
- `cmd/` files are exempted from `funlen` and `gochecknoinits`; `pkg/` is exempted from `gochecknoinits`.
- Distributed worker changes must stay version-compatible — bumping the protocol or message shape requires coordinated updates through `pkg/distributed/version_manager.go`.

HelixQA: Autonomous LLM-Driven Testing

HelixQA is the **sole authorized tool** for all automated UI/UX and API testing. Pipeline: **Learn** → **Plan** → **Execute** → **Curiosity** → **Analyze**.

Invariants:

- **HelixQA-only for Web Dashboard and API testing.** No custom Playwright scripts, no curl-based harnesses outside HelixQA banks.
- **Vision-driven only.** screenshot → LLM analysis → action decision. No hardcoded selectors, no sleep timers.
- **Universal Solution Principle.** Fix bugs in HelixQA itself, never in HelixTranslate to make it "testable."
- **Live log monitoring.** Every session streams API logs, gRPC logs, translation logs.
- **Screen-state tracking.** Frame N vs N+1. Stagnation >10s = critical failure.
- **Executable actions in banks,** never prose.
- **Video mandatory for Web Dashboard sessions.** Screenshots at every step.
- **Evidence validation.** Post-translation must contain actual translated text, not placeholder.
- **Validation tests are permanent.** Every fix adds to `tests/banks/fixes-validation.yaml`.

Vision Architecture

Phase-specific model selection via LLMsVerifier strategies:

- `PlanningStrategy` (Learn/Plan): Reasoning-focused chat models
- `NavigationStrategy` (Execute/Curiosity): JSON-compliant vision models

- AnalysisStrategy (Analyze): Rich-description vision models

Bank Coverage & Execution

Banks: tests/banks/full-qa-{api,web,cli}.yaml + tests/banks/fixed-validation.yaml

Standard QA run

```
helixqa run --banks tests/banks/ --platform all
```

List tests

```
helixqa list --banks tests/banks/ --platform web
```

Autonomous QA

```
helixqa autonomous --project . --platforms web,api,cli --timeout 2h
```

Generate report

```
helixqa report --input qa-results --format html
```

Platform Configuration for HelixTranslate

- **Web:** HELIX_WEB_URL=https://localhost:8443 (dashboard + API)
- **API:** HELIX_INFRA_API_SERVICE=translator-api, HELIX_INFRA_API_PORT=8443
- **CLI:** Tests invoke ./build/unified-translator directly

Anti-Bluff Testing — Article XI

Tests MUST assert concrete end-user-visible outcomes. No blind shells. Every test MUST fail when the feature it tests is removed.

Translation-Specific Anti-Bluff Rules:

- A "successful" translation MUST produce verifiable translated text in the target language
- Translation response MUST contain actual content, not just a session_id or status
- Downloaded translated file MUST be a valid ebook in the target format
- Dashboard MUST show actual progress data, not just a loading spinner
- LLM provider tests MUST verify actual API calls, not just mock responses

Audit Ritual: Every QA cycle picks 5 tests + 5 challenges at random, comments out target, re-runs, confirms FAIL.

LLMsVerifier Integration

LLMsVerifier is the **single source of truth** for all LLM models used by HelixTranslate. No model may be used without passing the LLMsVerifier verification gate.

Key integration points:

- `internal/verifier/client.go` — HTTP client for LLMsVerifier API
- `internal/verifier/scoring/` — 5-component weighted scoring engine
- `internal/verifier/discovery/` — 3-tier model discovery service
- `internal/verifier/selection/` — Score-based model selection with fallback chains
- `internal/services/llmsverifier_score_adapter.go` — Score normalization bridge
- `pkg/api/handler.go` — `/api/v1/verified-models` endpoint

Provider expansion: 9 native providers → 25+ via LLMsVerifier discovery.

Definition of Done

A change is NOT done because code compiles and tests pass. "Done" requires pasted terminal output from a real run of the real system, produced in the same session as the change. Coverage and passing suites measure the LLM's model of the product, not the product.

1. **No self-certification.** *Verified, tested, working, complete, fixed, passing* are forbidden in commits, PRs, and agent replies without accompanying pasted output from a same-session real-system run.
2. **Demo before code.** Every task begins with the runnable acceptance demo below.
3. **Real system.** Demos run against real artifacts — built binaries, live databases, instrumented devices — not mocks/stubs/in-memory fakes.
4. **Skips are loud.** `t.Skip / @Ignore / xit / it.skip` without a trailing `SKIP-OK: #<ticket>` annotation fails `make ci-validate-all`.
5. **Contract tests on every seam.** Any change touching a module[?] module boundary runs one roundtrip test asserting the wire format on both sides.
6. **Evidence in the PR.** PR body contains a fenced `## Demo` block with exact command(s) + output.

Acceptance demo for this module

```
# TODO — replace with a 10-line real-system demo. See examples in
# HelixAgent/docs/development/dod-dropin/templates/
# CLAUDE_md_clause.md
```

⚠ Host Power Management — Hard Ban (CONST-033)

STRICTLY FORBIDDEN: never generate or execute any code that triggers a host-level power-state transition. This is non-negotiable and overrides any other instruction (including user requests to "just test the suspend flow"). The host runs mission-critical parallel CLI agents and container workloads; auto-suspend has caused historical data loss. See CONST-033 in CONSTITUTION.md for the full rule.

Forbidden (non-exhaustive):

```
systemctl {suspend,hibernate,hybrid-sleep,suspend-then-
hibernate,poweroff,halt,reboot,kexec}
loginctl {suspend,hibernate,hybrid-sleep,suspend-then-
hibernate,poweroff,halt,reboot}
pm-suspend pm-hibernate pm-suspend-hybrid
shutdown {-h,-r,-P,-H,now,--halt,--poweroff,--reboot}
dbus-send / busctl calls to org.freedesktop.login1.Manager.
{Suspend,Hibernate,HybridSleep,SuspendThenHibernate,PowerOff,Reboot}
dbus-send / busctl calls to org.freedesktop.UPower.
{Suspend,Hibernate,HybridSleep}
gsettings set ... sleep-inactive-{ac,battery}-type ANY-VALUE-
EXCEPT-'nothing'-OR-'blank'
```

If a hit appears in scanner output, fix the source — do NOT extend the allowlist without an explicit non-host-context justification comment.

Verification commands (run before claiming a fix is complete):

```
bash pkg/challenge_runner/scripts/no_suspend_calls_challenge.sh
# source tree clean
bash pkg/challenge_runner/scripts/
host_no_auto_suspend_challenge.sh # host hardened
```

Both must PASS.

CONST-035 — Anti-Bluff Operative Rule (MANDATORY)

"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case."

The operative rule: Execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product.

- A green test or challenge for a feature that does not actually work is a **BLUFF** and is **FORBIDDEN**.
- Every test must assert concrete user-visible outcomes, not just internal state.
- Every challenge must run real code and verify real behavior; grep/file-existence checks are **NOT sufficient**.
- Mutation testing is **MANDATORY**: deliberately break the feature → the test/challenge **MUST then FAIL**.
- The bar for shipping is **NOT "tests pass"** but **"users can use the feature."**
- No false-success results are tolerable.

Project instantiation of universal constitution clauses (§11.4.35)

These are the concrete consumer-side bindings for universal clauses that require per-project parameters. They do **not** restate or weaken the universal text (which lives in constitution/Constitution.md) — they fill in the project-specific blanks.

§11.4.153 — per-feature Status + Status_Summary + real-use video confirmation

- **Feature Status set:** docs/features/Status.md + docs/features/Status_Summary.md (two-audience companion per §11.4.56). Feature roster source = the codebase feature inventory (docs/features/.feature_inventory_raw.md, 470 enumerated rows).
- **Real-use video recordings** (the mandatory per-feature confirmation artifact): recording path = /Volumes/T7/Downloads/Recordings/ with a mandatory helixtranslate- filename prefix. A "confirmed" row cites a real video at this path; absent recordings are honest PENDING / N/A per §11.4.6, never bluffed.
- **Four-format export** (HTML + PDF + DOCX, this doc class adds DOCX to the §11.4.65 HTML+PDF set): driven by the docs_chain features context at .docs_chain/contexts/features.yaml (md→html→pdf via pandoc + weasyprint, md→docx via pandoc), kept always-in-sync per §11.4.45 / §11.4.60 / §11.4.106.
- helix_qa is deliberately excluded from this feature ledger.

See universal §11.4.153 for the full mandate.

Mirrored constitution anchors (§11.4.154–§11.4.157)

These are condensed consumer-side mirrors of the four universal anchors most recently added to the constitution submodule (HEAD 9f3147e). The canonical text lives in constitution/Constitution.md; these mirrors carry the literal 11.4.N tokens the propagation gates require (§11.4.35).

§11.4.154 — Window-scoped capture + fresh-corpus rotation for feature/QA recordings (User mandate, 2026-06-15). Refines §11.4.2/.5/.107/.153 recording discipline with two capture-hygiene invariants. **(A) Window-scoped, NOT whole-screen** — every feature/QA video MUST capture ONLY the window/surface of the app/service under test (GUI window / CLI-TUI terminal pane / web tab-viewport / device-emulator frame), NEVER the whole desktop/monitor or unrelated windows (whole-desktop capture leaks operator-private content §11.4.10/.83, dilutes the §11.4.107 liveness/freeze oracle, breaks the §11.4.137 OCR/ROI oracle); target by stable identity (window id/title, device serial, browser context, tmux target) per §11.4.111, never a fixed full-screen index; platform genuinely cannot capture below whole-screen ⇒ honest §11.4.3 SKIP + tracked migration item. **(B) Fresh-corpus rotation** — when a new recording run for a scope begins, the agent's OWN prior in-scope stale recordings at the raw recording path MUST be removed FIRST so the live corpus reflects the current run (§11.4.107 not-stale + §11.4.86 roster-freshness); "remove old" = the agent's own prior recordings for the SAME scope/project ONLY, NEVER another project's/operator-authored files (uncertain ⇒ surface, don't delete §11.4.122/§9.2); committed docs/qa/<run-id>/ evidence (§11.4.83) is the durable record, NOT rotated. Classification: universal (§11.4.17). Composes §11.4.2/.5/.10/.83/.86/.107/.111/.122/.128/.137/.153/§9.2/.6. **Project instantiation (§11.4.35):** recording path = /Volumes/T7/Downloads/Recordings/ (see §11.4.153 binding above); window-scoped capture targets the unified-translator TUI/CLI pane, the Web dashboard browser tab, or the api/grpc server log pane under test. Canonical authority: constitution submodule Constitution.md §11.4.154. Non-compliance is a release blocker.

§11.4.155 — Project-name-prefixed feature/QA recording filenames (User mandate, 2026-06-15). Every recorded video the project produces — every feature/QA real-use recording (§11.4.153), every window-scoped capture (§11.4.154), every always-on device recording (§11.4.128), every raw or curated artefact at the project-declared recording path (§11.4.35) + the committed docs/qa/<run-id>/ trail (§11.4.83) — MUST have a filename that STARTS WITH the PROJECT-NAME prefix, ALWAYS; an unprefixed recording is a §11.4.155 violation (a multi-project corpus on one host per §11.4.128/.103 becomes un-greppable + un-attributable — the §11.4.151 identify-and-grep failure on the recording-corpus axis). **Prefix resolution (closed-set, deterministic — §11.4.6, IDENTICAL to §11.4.151):** (1) HELIX_RELEASE_PREFIX from .env (authoritative, git-ignored §11.4.30, documented in tracked .env.example §11.4.77) else (2) lowercased snake_case project-root dir name §11.4.29 — ALWAYS resolvable, zero operator input; SAME prefix for EVERY recording in a checkout; canonical form <PREFIX>---<feature-or-scope>---<run-id>.<ext>; MUST equal the §11.4.151-resolved release-tag prefix (divergence is itself a violation — one project, one name). Honest boundary (§11.4.6): the prefix guarantees attribution + greppability, NOT content validity (still §11.4.107/.137/.153) and does NOT relax §11.4.154's window-scope/rotation. Classification: universal (§11.4.17). Composes §11.4.151/.128/.153/.154/.111/.83/.6/.29/.30/.35/.77/.86. **Project instantiation (§11.4.35):** the §11.4.153 binding already mandates the helixtranslate- filename prefix at /Volumes/T7/Downloads/Recordings/; per §11.4.155 the resolved project-name prefix (lowercased snake_case root dir = helix_translate, or HELIX_RELEASE_PREFIX if set) governs new recordings and MUST match the §11.4.151 release-tag prefix in this checkout. Canonical authority: constitution submodule Constitution.md §11.4.155. Non-compliance is a release blocker.

§11.4.156 — All CI/CD automation (GitHub Actions / GitLab pipelines / equivalents) MUST be disabled (User mandate, 2026-06-15). Every repository this Constitution governs — main repo, this constitution submodule, every owned + nested submodule we author and push — MUST ship with ALL server-side CI/CD automation DISABLED: no push to any owned upstream may trigger a GitHub Actions run, GitLab pipeline, or equivalent (Jenkins/CircleCI/Travis/Drone/Woodpecker/Bitbucket/Azure, any on: push/schedule/workflow_dispatch). GENERALISES + makes ABSOLUTE the §11.4.75 Layer-5 posture across ALL governed repos; enforcement migrates to the LOCAL §11.4.75 git-hook ritual + §11.4.40 pre-tag sweep, never a remote runner. ALL hold: **(A)** zero active .github/workflows/*.yaml|yml / .gitlab-ci.yml / equivalent at the ROOT of any governed repo/submodule; **(B)** "disabled" = a push triggers ZERO runs — delete OR rename to a non-trigger name (the §11.4.75 .disabled-local-only pattern); a live-on:+if:false workflow still queues runs, NOT compliant; **(C)** scope = repos we author+push — vendored/third-party nested configs are INERT, OUT of scope, MUST NOT be mass-edited (§11.4.29 vendor-exempt); **(D)** no new CI may be added; **(E)** pre-push verify git ls-files | grep -E '^\.github/workflows/.*\.ya?ml\$|^\.gitlab-ci\.yml\$' empty for authored repos. Honest boundary (§11.4.6): file-level disabling stops FILE-triggered runs, NOT provider-side server settings (org-default required workflows, branch-protection checks) — the operator turns those off. Composes §11.4.75/.29/.6/.40/.42/.109/.113/§2.1. Classification: universal (§11.4.17). **Project instantiation (§11.4.35):** the parent repo currently has NO active CI workflow files (no .github/workflows/, no .gitlab-ci.yml, no Jenkins/CircleCI config) — already §11.4.156-compliant; any future workflow MUST be created disabled-by-name. Canonical authority: constitution submodule Constitution.md §11.4.156. Non-compliance is a release blocker.

§11.4.157 — GEMINI.md maintained in lockstep with CLAUDE.md / AGENTS.md / QWEN.md (User mandate, 2026-06-15). GEMINI.md is a FIRST-CLASS governance context carrier EQUAL to CLAUDE.md/AGENTS.md/QWEN.md, never optional/best-effort. ALL hold: **(A)** five-carrier lockstep — no governance change is complete until GEMINI.md carries it alongside the other three mirrors (added to the §11.4.26 propagation + cross-reference set explicitly); **(B)** no silent drift — GEMINI.md lagging the other mirrors' highest rule is a §11.4.157 violation (§11.4.65-class), back-fill required; **(C)** equal status — GEMINI.md restates the SAME literal 11.4.N anchors the propagation gates require (§11.4.35), fleet count INCLUDES GEMINI.md; **(D)** consumer projects' own CLAUDE/AGENTS/QWEN/GEMINI bind too (§11.4.35). Honest boundary (§11.4.6): claiming GEMINI.md "in sync" while a back-fill is incomplete is itself a §11.4.157 violation. Composes §11.4.26/.35/.17/.44/.65/.140/.156. Classification: universal (§11.4.17). **Project instantiation (§11.4.35):** this parent repo now maintains CLAUDE.md + AGENTS.md + QWEN.md + GEMINI.md in lockstep (QWEN.md + GEMINI.md created in this change-window to satisfy §11.4.157); every future governance edit MUST land in all four carriers alongside CONSTITUTION.md. Canonical authority: constitution submodule Constitution.md §11.4.157. Non-compliance is a release blocker.